

Teste Técnico 01 - Desenvolvedor(a) C# (.NET) Júnior

Prazo de Entrega: 3 dias a contar do recebimento deste documento.

Formato de Entrega:

- Link para repositório público no GitHub (ou GitLab) contendo a solução e a documentação solicitada.
- Encaminhar link para:
 - sabrina.koester@havan.com.br.
 - rafael.henrique@havan.com.br.

Orientações Gerais e Política do Teste

Uso de Ferramentas e IA: Sabendo do cenário atual, você **pode** utilizar a internet ou assistentes de IA para consulta de sintaxe ou conceitos. **Contudo**, a avaliação final dependerá integralmente da sua capacidade de **explicar e defender** cada linha de código e decisão tomada.

Histórico de Commits (Obrigatório): Não aceite fazer um único *commit* com o código pronto no final. Faça *commits* frequentes e descritivos à medida que avança na resolução. Queremos acompanhar seu processo de raciocínio através do Git.

Seção "Defesa do Código": O arquivo 'README.md' do seu repositório deve ser escrito no formato *Markdown* e conter respostas detalhadas para as perguntas de defesa listadas no final deste teste.

Parte 1: Questão Teórica

Questão 1: Conceitos de Orientação a Objetos e Runtime do .NET

(Insira suas respostas diretamente na seção correspondente do `README.md`)

Explique com suas próprias palavras os seguintes conceitos no contexto da plataforma .NET:

- Qual é a diferença fundamental entre **Classes (Value Types vs Reference Types)** em C# e como isso afeta o uso de memória (Stack vs Heap)?
- Explique a diferença entre usar 'Interface' e 'Classe Abstrata'. Dê um exemplo prático de quando escolheria uma em detrimento da outra.
- O que é e para que serve o operador 'async/await'? O que acontece na prática quando uma thread do .NET executa uma operação assíncrona?

Parte 2: Lógica de Programação

(Você pode resolver estas questões na linguagem C# através de uma Console Application, demonstrando a saída de cada exercício no console).

Questão 2: Validador de Sequências e Agrupamento

Escreva uma função em C# que receba uma lista de inteiros desordenados e retorne a maior sequência de números inteiros **consecutivos** encontrados na lista.

Exemplo de Entrada: '[100, 4, 200, 1, 3, 2]'

Saída Esperada: '[1, 2, 3, 4]' (Tamanho 4)

Regra: A ordem original dos elementos no array não importa para formar a sequência consecutiva. Tente pensar na eficiência da sua solução.

Questão 3: Análise de String e Frequência

Crie um método em C# que receba uma frase (string) e realize as seguintes análises:

1. Higienização e Normalização:

- Remova espaços, pontuações e caracteres especiais.
- Ignore a diferença entre letras maiúsculas e minúsculas (trate tudo como minúsculo).
- (Opcional/Bônus): Ignore acentuações (ex: tratar 'á' como 'a').

2. Primeiro Caractere Não Repetido:

- Encontre o primeiro caractere da frase original que aparece exatamente **uma única vez** após a higienização.

3. Top 3 Caracteres Mais Frequentes:

- Liste os 3 caracteres que mais se repetem na frase, informando a letra e a quantidade exata de ocorrências.

Exemplo Prático:

- Entrada:** "A Bateria do computador está Fraca!"
- Texto Higienizado:** "abateriadocomputadorestafraca"
- Saída Esperada no Console:**
 - **Primeiro caractere não repetido:** 'i'
 - **Top 3 caracteres mais frequentes:**
 - Letra 'a': 6 vezes
 - Letra 'o': 3 vezes
 - Letra 'e': 3 vezes (ou outra letra com 3 ocorrências, como 't')*

Questão 4: Processamento Financeiro e Regra de Negócio

Imagine que você precisa processar uma lista de pagamentos com as seguintes regras de juros e descontos acumulados por dia de atraso/antecipação:

- Valor Base da Fatura:** R\$ 1.000,00.

- **Data de Vencimento:** 10/10/2026.

Regras:

- Pagamento feito **antes da data**: Desconto de 1% por dia de antecipação (máximo de 10% de desconto).
- Pagamento feito **na data**: Valor original.
- Pagamento feito **após a data**: Multa fixa de 2% + Juros simples de 0,5% por dia de atraso.

Crie um método que receba a **data do pagamento efetuado** e retorne o **valor final calculado**, exibindo também o detalhamento (se houve multa, juros ou desconto).

Parte 3: Questão de Programação Prática

Questão 5: Mini-API / Console App de Gerenciamento de Tarefas (To-Do List)

Crie uma aplicação em C# (pode ser uma **Web API minimal em ASP.NET Core** ou uma **Console Application com arquitetura em camadas**) para gerenciamento de tarefas pessoais.

Requisitos Funcionais:

1. Cadastro de Tarefa:

- a. Propriedades obrigatórias: `Id` (GUID ou int), `Título`, `Descrição`, `DataDeCriacao`, `DataDeConclusao` (múltiplos valores nulos até ser concluída), e `Status` (Pendente, EmAndamento, Concluída).

2. Atualização de Status:

- a. Permitir alterar o status de uma tarefa. Se o status mudar para `Concluída`, a `DataDeConclusao` deve ser preenchida automaticamente com o horário atual.

3. Filtros e Consultas:

- a. Listar todas as tarefas ativas (Pendente / EmAndamento). Filtrar tarefas concluídas em um intervalo de datas específico.

4. Validações de Negócio:

- a. O `Título` não pode ser vazio e deve ter no mínimo 5 caracteres.
- b. Não deve ser permitido alterar o status de uma tarefa que já esteja `Concluída`.

Requisitos Técnicos:

- Respeitar conceitos de Orientação a Objetos (Separação de responsabilidades / Domain/Services/Controllers ou Handlers).
- Tratamento adequado de exceções e retornos HTTP (caso opte por API) ou mensagens claras ao usuário (caso opte por Console App).
- Uso de Persistence In-Memory (pode usar listas dinâmicas em memória ou `Entity Framework Core In-Memory`).

Questão Bônus: Interface Web em React (Frontend Integration)

Nota: Esta questão é **opcional** e contará como pontuação extra/diferencial para a vaga.

Caso você tenha conhecimentos de frontend e tenha implementado a Questão 5 como uma **Web API**, crie uma aplicação SPA (Single Page Application) simples utilizando **React** (TypeScript ou JavaScript) que consuma a sua API C#.

Requisitos da Tela:

1. Formulário de Criação:

- a. Um formulário simples para cadastrar uma nova tarefa (Título e Descrição).
- b. Exibir mensagens de validação retornadas pelo backend (ex: erro se o título tiver menos de 5 caracteres).

2. Listagem de Tarefas:

- a. Exibir as tarefas em formato de lista ou cartões (cards).
- b. Exibir visualmente o status atual de cada tarefa (ex: uso de tags ou cores diferentes para *Pendente*, *Em Andamento* e *Concluída*).

3. Interatividade:

- a. Botão para alternar ou avançar o status da tarefa (ex: mover de *Pendente* para *Em Andamento*, e de *Em Andamento* para *Concluída*).
- b. Atualizar a listagem automaticamente após cadastrar ou alterar uma tarefa.

Requisitos Técnicos do Bônus:

- A aplicação React pode ser criada dentro de uma pasta no mesmo repositório (ex: `/frontend` ou `/client`).
- Uso de bibliotecas de requisição HTTP como `axios` ou a API nativa `fetch`.
- Não é necessário foco excessivo em design complexo ou CSS avançado; o objetivo é avaliar a **integração entre o React e a API C#**, manipulação de estado (`useState`, `useEffect`) e tratamento de erros na tela.

Parte 4: Defesa do Código e Entrevista Técnica

No arquivo `README.md` do seu projeto, você deve incluir obrigatoriamente a seção "**Defesa da Solução**", respondendo com clareza aos seguintes tópicos:

1. Decisão de Arquitetura:

- a. Como você organizou as pastas/projetos da Questão 5?
- b. Por que escolheu essa estrutura?

2. Desafios e Soluções:

- a. Qual foi a parte mais complexa da lógica nas Questões 2, 3 ou 4?
- b. Como você resolveu e testou esse trecho?

3. Autoavaliação e Trade-offs:

- a. Se você tivesse mais tempo para desenvolver este teste, o que mudaria ou melhoraria na sua implementação (ex: testes unitários, banco de dados real, logs, etc.)?

4. Pergunta de Checagem Técnica:

- a. Explique passo a passo o que acontece no seu código da **Questão 2** a partir do momento em que o array desordenado entra na função até a entrega da resposta final.

Atenção!!!

Na etapa seguinte do processo seletivo (entrevista presencial ou por vídeo), faremos perguntas diretamente baseadas nas suas respostas do README e solicitaremos que você faça uma pequena alteração ou refatoração ao vivo no seu código.